

ML ACADEMY

Machine Learning Interview Cheatsheet

100 Essential Questions & Answers
for FAANG & Top Tech Company Interviews

Foundations • Regression • Classification • Ensemble Methods
Unsupervised Learning • Evaluation • Feature Engineering
Deep Learning • Python & Libraries • System Design

www.learnmlacademy.com

Free Resource — Share with your network

How to get the most from this cheatsheet

This PDF contains 100 carefully curated ML interview questions across 10 sections. Each answer is concise enough for quick revision but detailed enough to deliver in a real interview. **Recommended approach:** read the question, try to answer it yourself first, then compare with the provided answer. Focus extra time on sections where your answer was incomplete.

For coding questions: always clarify requirements first, state your approach before coding, discuss time and space complexity, and mention edge cases. **For system design:** discuss tradeoffs rather than claiming one approach is always best.

Section	Questions
1. Foundations of Machine Learning	10
2. Regression Algorithms	5
3. Classification Algorithms	9
4. Ensemble Methods	5
5. Unsupervised Learning	5
6. Model Evaluation & Tuning	6
7. Feature Engineering & Preprocessing	5
8. Deep Learning	8
9. Python & ML Libraries	5
10. ML System Design & Production	7
11. Statistics & Probability for ML	5
12. Advanced ML Topics	10
13. Time Series & Specialised Topics	5
14. Interview Strategy & Coding	5
15. Bonus: Real Interview Questions from Top Companies	10
TOTAL	100

1. Foundations of Machine Learning

Q1

What is Machine Learning? How does it differ from traditional programming?

Answer:

Traditional programming: Data + Rules → Answers. Machine Learning: Data + Answers → Rules. In ML, the algorithm discovers patterns and builds its own rules from examples. Traditional programs need every rule explicitly coded; ML systems generalise from data to handle unseen situations automatically.

Q2

What are the three main types of Machine Learning?

Answer:

Supervised Learning: Model trains on labelled input-output pairs to predict outputs for new inputs. Examples: spam detection, house price prediction. Unsupervised Learning: No labels — model finds hidden structure in raw data. Examples: customer segmentation, anomaly detection. Reinforcement Learning: An agent learns by taking actions in an environment and receiving rewards or penalties. Examples: game playing (AlphaGo), robotics.

Q3

What is the difference between classification and regression?

Answer:

Classification predicts a discrete category or class label. Examples: Is this email spam or not? What digit is in this image? Output is one of a finite set of classes. Regression predicts a continuous numerical value. Examples: What will the house price be? What will tomorrow's temperature be? Output is a real number. Key metric for classification: accuracy, F1. For regression: MSE, RMSE, MAE, R-squared.

Q4

What is the Bias-Variance Tradeoff? Why does it matter?

Answer:

Bias = error from oversimplified assumptions (model underfits — misses relevant patterns). Variance = error from oversensitivity to training data noise (model overfits — learns noise as signal). Total error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$. The tradeoff: increasing model complexity reduces bias but increases variance. A simple linear model has high bias; a deep decision tree has high variance. The goal is finding the complexity sweet spot with low bias AND low variance.

Q5

Explain overfitting and underfitting with examples.

Answer:

Overfitting: Model memorises training data including noise. Performs excellently on training set (99% accuracy) but poorly on unseen data (60% accuracy). Example: a decision tree with no `max_depth` that perfectly separates every training point. Underfitting: Model is too simple to capture patterns. Performs poorly on both training and test sets. Example: fitting a straight line to clearly non-linear data. Diagnosis: plot learning curves — if train accuracy >> test accuracy, overfitting. If both are low, underfitting.

Q6

What is the difference between a model parameter and a hyperparameter?

Answer:

Parameters are learned FROM data automatically during training. Examples: weights and biases in a neural network, coefficients in linear regression, split thresholds in a decision tree. They define the model itself. Hyperparameters are set BY the practitioner BEFORE training and control the learning process. Examples: learning rate, number of trees in Random Forest, max_depth, number of hidden layers, dropout rate, regularisation strength (C in SVM, alpha in Ridge). Hyperparameters require tuning via cross-validation or grid search.

Q7

What is cross-validation? Why is it better than a single train/test split?

Answer:

K-Fold Cross-Validation: Split data into K equal folds. For each fold: train on the other K-1 folds, evaluate on this fold. Average the K scores. Benefits over single split: (1) Every sample is used for both training and validation, (2) Less sensitive to lucky/unlucky splits, (3) Gives confidence interval on performance, (4) More reliable overfitting detection, (5) Uses all available data efficiently. Standard practice: 5-fold or 10-fold CV. Stratified K-Fold preserves class proportions in each fold — essential for imbalanced datasets.

Q8

What is the curse of dimensionality?

Answer:

As the number of features (dimensions) increases exponentially, the volume of feature space grows so large that data becomes extremely sparse. Consequences: (1) Distance metrics lose meaning — in high dimensions all points become equidistant, breaking KNN. (2) Exponentially more data needed to maintain statistical significance. (3) Models overfit more easily. (4) Visualisation becomes impossible. Solutions: Feature selection (remove irrelevant features), PCA/dimensionality reduction, regularisation, domain knowledge to engineer fewer better features.

Q9

What is data leakage? Give two examples.

Answer:

Data leakage occurs when information from outside the training data — typically from the test set or future data — influences model training, causing unrealistically optimistic evaluation metrics that do not generalise. Example 1: Fitting StandardScaler on the full dataset before splitting. The scaler learns statistics from test data, giving the model information it should not have. Fix: always fit preprocessing inside a Pipeline that is trained only on training data. Example 2: Including a feature like 'loan_default_flag' when predicting loan defaults — this directly encodes the target. Fix: careful feature audit before training.

Q10

Explain the concept of feature engineering.

Answer:

Feature engineering is the process of using domain knowledge to transform raw data into features that better represent the underlying patterns for ML algorithms. Techniques: (1) Polynomial features — $x_1 \times x_2$ captures interactions. (2) Binning — convert continuous salary into Low/Mid/High bands. (3) Log transform — reduce skew in right-skewed distributions like income. (4) Date decomposition — extract hour, day_of_week, is_weekend from timestamps. (5) Aggregation — mean transaction value per customer. (6) Encoding — one-hot encode categorical variables. Good feature engineering often matters more than algorithm choice.

2. Regression Algorithms

Q11

How does Linear Regression work? What are its assumptions?

Answer:

Linear Regression fits a hyperplane $y = w_0 + w_1x_1 + \dots + w_nx_n$ that minimises the sum of squared residuals (Ordinary Least Squares). The optimal weights are found analytically: $w = (X^T X)^{-1} X^T y$, or via gradient descent for large datasets. Assumptions: (1) Linear relationship between features and target, (2) Independence of observations (no autocorrelation), (3) Homoscedasticity — constant variance of residuals, (4) Normally distributed residuals, (5) No multicollinearity between features. Violating assumptions does not break the algorithm but makes coefficient estimates unreliable.

Q12

What is the difference between Ridge and Lasso regression?

Answer:

Both add a regularisation penalty to the OLS loss to prevent overfitting. Ridge (L2): $\text{Loss} = \text{OLS} + \alpha \sum (w_i^2)$. Penalises large weights but shrinks them towards zero without reaching zero. Keeps all features. Best when many features contribute modestly. Lasso (L1): $\text{Loss} = \text{OLS} + \alpha \sum |w_i|$. Produces sparse solutions — drives some weights to exactly zero, effectively performing feature selection. Best when you suspect most features are irrelevant. Elastic Net combines both: $\text{Loss} = \text{OLS} + \alpha \cdot (\text{L1_ratio} \cdot \text{L1} + (1 - \text{L1_ratio}) \cdot \text{L2})$. Use when you want both sparsity and stability.

Q13

What is gradient descent? Explain Batch, Stochastic and Mini-batch variants.

Answer:

Gradient descent iteratively updates parameters by moving in the negative gradient direction: $w = w - \text{learning_rate} \cdot \text{gradient}(\text{Loss})$. Batch GD: Computes gradient using ALL training samples per update. Stable convergence but very slow for large datasets. Stochastic GD (SGD): Uses ONE random sample per update. Very fast but noisy — loss bounces around. Mini-batch GD: Uses a small batch (32-256 samples). Best of both worlds — fast, reasonably stable. Industry standard used by PyTorch and TensorFlow. The learning rate controls step size — too large: overshoots minimum. Too small: converges very slowly.

Q14

What is polynomial regression? What is the risk?

Answer:

Polynomial regression extends linear regression by adding polynomial terms of features: $y = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots$. This allows fitting non-linear relationships while still using the linear regression framework (the model is linear in its parameters, just not in x). Risk: overfitting. A degree-20 polynomial can perfectly fit 20 training points while wildly oscillating between them — useless for prediction. Always use cross-validation to select degree and regularisation to control complexity.

Q15

How would you evaluate a regression model?

Answer:

MSE (Mean Squared Error): average of squared differences between predicted and actual. Penalises large errors heavily. RMSE (Root MSE): same as MSE but in the same units as the target — more interpretable. MAE (Mean Absolute Error): average of absolute differences. More robust to outliers than MSE. R-squared (coefficient of determination): proportion of variance in target explained by the model. $R^2 = 1$ means perfect fit, $R^2 = 0$ means model is no better than predicting the mean, negative R^2 means model is worse than the mean. Adjusted R^2 penalises for adding useless features.

3. Classification Algorithms

Q16

How does Logistic Regression work?

Answer:

Logistic Regression models the probability of a binary outcome. It computes a linear combination $z = w_0 + w_1x_1 + \dots + w_nx_n$ then passes it through the sigmoid function: $P(y=1|x) = 1 / (1 + e^{-z})$. Output is a probability between 0 and 1. Classification threshold is typically 0.5 but can be adjusted. Training minimises binary cross-entropy loss using gradient descent. Despite the name, it is a classification algorithm. Multiclass extension: Softmax Regression (one-vs-rest or multinomial logistic regression).

Q17

Explain how a Decision Tree makes decisions. What is information gain?

Answer:

At each node a decision tree asks a yes/no question about one feature. It selects the question that best separates the classes. Information Gain = Entropy(parent) - weighted average Entropy(children). Entropy = $-\sum(p * \log_2(p))$ measures impurity. Alternatively, Gini Impurity = $1 - \sum(p^2)$ is faster to compute and gives similar results. The tree recursively splits until it reaches a pure node (all one class) or hits a stopping criterion (max_depth, min_samples_split). Decision trees are highly interpretable and require no feature scaling.

Q18

What is Random Forest? How does randomness help?

Answer:

Random Forest builds N decision trees (typically 100-500), each trained on a bootstrap sample (random sample with replacement) of the training data. At each node split, only a random subset of $\sqrt{n_{\text{features}}}$ features is considered. Predictions are aggregated by majority vote (classification) or mean (regression). Randomness helps by: (1) Bootstrap sampling creates diverse training sets for each tree (reduces variance). (2) Random feature selection prevents all trees from making the same splits on dominant features — reduces correlation between trees. Correlated trees do not reduce variance when averaged.

Q19

How does Naive Bayes classification work? Why is it called naive?

Answer:

Naive Bayes applies Bayes theorem: $P(\text{class}|\text{features})$ proportional to $P(\text{features}|\text{class}) * P(\text{class})$. It is called naive because it assumes ALL features are conditionally independent given the class — an assumption rarely true in practice. Despite this, it performs surprisingly well for text classification. Gaussian NB assumes features follow a normal distribution — used for continuous data. Multinomial NB uses word count frequencies — ideal for text. Bernoulli NB uses binary features (word present or absent). Key advantage: extremely fast training, works well with small datasets and high-dimensional sparse data.

Q20

Explain the Support Vector Machine algorithm.

Answer:

SVM finds the hyperplane that maximises the margin between classes. The margin is the distance between the hyperplane and the nearest data points from each class (support vectors). Maximising the margin improves generalisation. Soft-margin SVM (parameter C): allows some misclassifications. High C = hard margin, may overfit. Low C = soft margin, more generalisable. The kernel trick implicitly maps features to higher dimensions enabling non-linear classification. RBF (Gaussian) kernel: $K(x,y) = \exp(-\gamma * ||x-y||^2)$ is the most common. SVM needs feature scaling. Does not scale well to very large datasets.

Q21

How does K-Nearest Neighbors work? What are its limitations?

Answer:

KNN stores all training data. To classify a new point: (1) Calculate distance (usually Euclidean) to all training points, (2) Find K nearest neighbours, (3) Take majority vote of their labels. It is a lazy learner — no training phase. Limitations: (1) Prediction is $O(n)$ per query — very slow for large datasets, (2) Performance degrades with irrelevant features (they add noise to distance), (3) Must scale features — unscaled features dominate distance, (4) High memory usage — stores all training data, (5) Struggles in high dimensions (curse of dimensionality), (6) Choosing K requires tuning — odd K avoids ties.

Q22

What is the ROC curve and AUC score?

Answer:

ROC (Receiver Operating Characteristic) curve plots True Positive Rate (Recall) on Y axis vs False Positive Rate ($FPR = FP/(FP+TN)$) on X axis at every possible classification threshold from 0 to 1. AUC (Area Under the ROC Curve) summarises the curve in a single number. AUC = 1.0: perfect classifier. AUC = 0.5: random classifier (diagonal line). AUC = 0.8+: good. AUC is threshold-independent — useful when you need to compare models before deciding on a threshold. Advantage: works well for imbalanced classes. Use PR-AUC instead of ROC-AUC when positive class is very rare.

Q23

What is the confusion matrix? Define TP, FP, FN, TN, Precision, Recall, F1.

Answer:

Confusion Matrix: rows = actual class, columns = predicted class. TP (True Positive): predicted positive, actually positive. FP (False Positive / Type I error): predicted positive, actually negative. FN (False Negative / Type II error): predicted negative, actually positive. TN (True Negative): predicted negative, actually negative. Precision = $TP/(TP+FP)$: of all predicted positives, what fraction are correct? Recall = $TP/(TP+FN)$: of all actual positives, what fraction did we catch? $F1 = 2*P*R/(P+R)$: harmonic mean — use when you need to balance precision and recall. Accuracy = $(TP+TN)/Total$: misleading on imbalanced datasets.

Q24

When would you choose Precision over Recall as your primary metric?

Answer:

Choose RECALL (minimise false negatives) when missing a positive is the worse error: cancer/disease detection (missing cancer = untreated patient), fraud detection (missing fraud = financial loss), safety-critical systems. Choose PRECISION (minimise false positives) when false alarms are the worse error: email spam filter (legitimate emails in spam = frustrated users), recommendation systems (irrelevant recommendations = bad UX), content moderation (false removal of legitimate content). Use F1 when neither error type is clearly more costly. In practice: always discuss the business cost of each error type before choosing.

4. Ensemble Methods

Q25

What is ensemble learning? What problem does it solve?

Answer:

Ensemble learning combines predictions from multiple models to produce a better prediction than any individual model. It exploits the wisdom of the crowd — independent models make different errors, and combining them averages out those errors. Three main paradigms: Bagging (Random Forest) — trains models in parallel on random data subsets, reduces variance. Boosting (XGBoost, AdaBoost) — trains models sequentially, each correcting previous errors, reduces bias. Stacking — trains a meta-model on the predictions of base models, reduces both. Condition for improvement: base models must be diverse (making different errors) and individually better than random.

Q26

What is AdaBoost? How does it update sample weights?

Answer:

AdaBoost (Adaptive Boosting) trains weak classifiers sequentially. After each classifier, samples that were misclassified receive higher weights so the next classifier focuses on them. Process: (1) Initialise equal weights for all samples. (2) Train a weak classifier (usually a depth-1 decision stump). (3) Calculate its error rate. (4) Compute classifier weight $\alpha = 0.5 * \ln((1-\text{error})/\text{error})$ — better classifiers get higher α . (5) Increase weights of misclassified samples, decrease weights of correctly classified ones. (6) Repeat. Final prediction: weighted vote of all classifiers. Sensitive to noisy data and outliers.

Q27

Explain Gradient Boosting and how it differs from AdaBoost.

Answer:

Gradient Boosting fits each new tree to the RESIDUAL ERRORS (negative gradients of the loss function) of the current ensemble, rather than upweighting misclassified samples like AdaBoost. Algorithm: F_0 = initial prediction (mean). For $t=1$ to T : compute residuals $r_t = y - F_{t-1}(x)$, fit a tree h_t to r_t , update $F_t = F_{t-1} + \text{learning_rate} * h_t$. The learning rate (shrinkage) prevents overfitting. Differences from AdaBoost: works with any differentiable loss function, not just classification. More flexible, generally more accurate. Slower but more tunable.

Q28

What is XGBoost? Why is it so popular in competitions?

Answer:

XGBoost (Extreme Gradient Boosting) is an optimised implementation of gradient boosting. Improvements over vanilla gradient boosting: (1) Regularisation terms (L1 and L2) in the objective prevent overfitting. (2) Second-order Taylor expansion of loss for better tree building. (3) Column subsampling (like Random Forest) adds diversity. (4) Built-in handling of missing values. (5) Parallel processing for tree construction. (6) Cache-aware access patterns for speed. Why popular: combines high accuracy with speed, handles mixed data types, has extensive hyperparameters for tuning, wins many Kaggle competitions, production-ready.

Q29

What is the difference between hard and soft voting in ensemble classifiers?

Answer:

Hard voting: each model votes for a class label, final prediction is the majority class. Simple and effective when models have similar accuracy. Soft voting: each model outputs class probabilities, final prediction is the class with the highest average probability across all models. Generally better than hard voting because: it uses confidence information (a model that is 99% sure contributes more than one that is 51% sure), and tends to produce better calibrated probabilities. Requirement: all base models must support `predict_proba` (probability estimates).

5. Unsupervised Learning

Q30

Explain K-Means clustering. What are its limitations?

Answer:

K-Means partitions data into K clusters by iteratively: (1) Assigning each point to the nearest centroid (Euclidean distance), (2) Recomputing centroids as the mean of all assigned points. Continues until assignments stabilise. Choosing K: Elbow method (plot inertia vs K, pick the elbow), Silhouette score (higher is better, range -1 to 1). Limitations: (1) Must specify K in advance, (2) Assumes spherical, similarly-sized clusters, (3) Sensitive to outliers (they pull centroids), (4) Non-deterministic — different initialisation can give different results (use k-means++), (5) Fails with non-convex cluster shapes (use DBSCAN instead).

Q31

What is PCA? Explain step by step how it works.

Answer:

PCA (Principal Component Analysis) reduces dimensionality by finding axes of maximum variance. Steps: (1) Standardise features (subtract mean, divide by std). (2) Compute the covariance matrix of features. (3) Compute eigenvectors and eigenvalues of the covariance matrix. (4) Sort eigenvectors by eigenvalue (highest = most variance). (5) Select top K eigenvectors as principal components. (6) Project data onto these K components. Each PC is a linear combination of original features. The first PC captures the most variance; each subsequent PC is orthogonal to previous ones and captures decreasing variance. `explained_variance_ratio_` shows how much variance each component explains.

Q32

What is DBSCAN? How does it differ from K-Means?

Answer:

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) defines clusters as dense regions of points separated by sparse regions. Parameters: `eps` (neighbourhood radius), `min_samples` (minimum points to form a core point). Algorithm: (1) A core point has at least `min_samples` neighbours within `eps` distance. (2) Border points are within `eps` of a core point but have fewer than `min_samples` neighbours. (3) Noise points are neither core nor border. Differences from K-Means: (1) Does not require specifying K, (2) Discovers arbitrarily shaped clusters, (3) Explicitly handles noise and outliers, (4) Fails on clusters of varying density.

Q33

What is the Apriori algorithm? What are support, confidence and lift?

Answer:

Apriori finds frequent itemsets and association rules in transaction data. $\text{Support}(A) = \text{transactions containing } A / \text{total transactions}$ — how frequently A appears. $\text{Confidence}(A \rightarrow B) = \text{support}(A \text{ and } B) / \text{support}(A)$ — given A, how often does B appear? $\text{Lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$ — $\text{lift} > 1$ means A and B appear together more than expected by chance. Algorithm: start with 1-itemsets, prune those below `min_support` threshold, generate larger itemsets from frequent ones (Apriori principle: all subsets of a frequent itemset must be frequent). Generate rules from frequent itemsets filtered by `min_confidence`. Classic use case: market basket analysis (customers who buy bread and butter also buy milk).

Q34

What is t-SNE? When should you use it vs PCA?

Answer:

t-SNE (t-Distributed Stochastic Neighbour Embedding) is a non-linear dimensionality reduction technique primarily used for 2D/3D visualisation of high-dimensional data. It converts Euclidean distances between points into probabilities and minimises the KL divergence between high-dimensional and low-dimensional probability distributions. Use t-SNE when: you want to visualise clusters in 2D/3D, data has non-linear structure. Use PCA when: you need linear dimensionality reduction for downstream ML models, you need interpretable components, you need reproducible results (t-SNE is non-deterministic), or you have very large datasets (t-SNE is $O(n^2)$ — slow). Key: t-SNE is for visualisation only, not for feeding features to models.

6. Model Evaluation & Tuning

Q35

What is L1 vs L2 regularisation? When would you use each?

Answer:

Both add a penalty term to the loss function to discourage large weights and prevent overfitting. L1 (Lasso): $\text{penalty} = \alpha * \sum(|w_i|)$. Produces SPARSE solutions — drives some weights to exactly zero, performing automatic feature selection. Use when you suspect many features are irrelevant or want an interpretable model with few features. L2 (Ridge): $\text{penalty} = \alpha * \sum(w_i^2)$. Shrinks all weights towards zero but rarely to exactly zero. Keeps all features but reduces their influence. Better when all features contribute somewhat. Elastic Net = $L1_ratio * L1 + (1 - L1_ratio) * L2$: combines both, useful when features are correlated.

Q36

What is hyperparameter tuning? Explain Grid Search vs Random Search.

Answer:

Hyperparameter tuning finds the hyperparameter values that maximise model performance on validation data. Grid Search: exhaustively tries every combination of specified values. Guaranteed to find the best combination within the grid but exponentially expensive as dimensions increase. Random Search: samples random combinations for a fixed number of iterations. Surprisingly effective — often finds near-optimal parameters faster because most hyperparameters have small effects and random search covers the important dimensions more efficiently. Bayesian Optimisation (Optuna, Hyperopt): builds a probabilistic model of the objective function and intelligently selects next parameters to evaluate. Most efficient for expensive models.

Q37

What is the difference between validation set and test set?

Answer:

Training set: data the model learns from. Validation set: held-out data used DURING development to tune hyperparameters, compare models, and make decisions. The model never trains on this data, but decisions are made based on its performance, so it indirectly influences the model. Test set: completely held-out data used ONLY ONCE at the very end to estimate real-world performance. It must never influence any decisions. Common mistake: using test set performance to select models or tune hyperparameters — this causes the test set to effectively become a validation set, giving optimistic performance estimates.

Q38

What is stratified sampling? When is it important?

Answer:

Stratified sampling ensures each split (train/val/test) contains the same proportion of each class as the full dataset. Critical for: imbalanced classification (if 95% of data is class 0 and 5% is class 1, a random split might put all class 1 examples in training). In sklearn: `train_test_split(X, y, stratify=y)` and `StratifiedKFold` ensure proportional representation. Also important for time series: do not use random splitting — use `TimeSeriesSplit` to respect temporal ordering and prevent future data leaking into training.

Q39

How would you handle class imbalance in a dataset?

Answer:

Class imbalance: one class has far more samples than another (e.g. 99% fraud-free, 1% fraud). Solutions: (1) Resampling: oversample minority class (SMOTE creates synthetic minority samples) or undersample majority class. (2) Class weights: pass `class_weight='balanced'` to sklearn classifiers — penalises misclassifying minority class more heavily. (3) Threshold adjustment: lower classification threshold to increase recall for minority class. (4) Use appropriate metrics: not accuracy (misleading) — use F1, ROC-AUC, Precision-Recall AUC. (5) Ensemble methods: `EasyEnsemble`, `BalancedRandomForest`. Start with class weights — simple, effective, no data distortion.

Q40

What is early stopping in machine learning?

Answer:

Early stopping monitors validation loss during training and stops when it stops improving, preventing overfitting. Process: after each epoch, evaluate validation loss. If validation loss has not improved for 'patience' epochs, stop training and restore the weights from the best epoch. Benefits: (1) Prevents overfitting without explicit regularisation, (2) Reduces training time, (3) Acts as a form of regularisation. In Keras: `EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)`. In XGBoost: `early_stopping_rounds=50`. Also useful for neural networks where training loss continues decreasing but validation loss starts increasing — the classic overfitting signature.

7. Feature Engineering & Preprocessing

Q41

How do you handle missing data in a dataset?

Answer:

Step 1: Understand WHY data is missing. Missing Completely At Random (MCAR) — safe to drop. Missing At Random (MAR) — impute. Not Missing At Random (NMAR) — the missingness itself is informative, create indicator column. Techniques: (1) Drop rows — only when data is abundant and missing is MCAR. (2) Mean/Median imputation — mean for symmetric, median for skewed distributions. (3) Mode imputation — for categorical columns. (4) Forward/Backward fill — for time series. (5) KNN imputation — uses similar samples. (6) Multiple imputation — runs several imputations and averages. Always: create a binary 'was_missing' indicator feature before imputing to preserve the information that data was missing. Always fit imputer on training data only.

Q42

What is one-hot encoding vs label encoding? When to use each?

Answer:

Label Encoding: converts categories to integers (Red=0, Blue=1, Green=2). Use only for ordinal categories with a natural order (Small=0, Medium=1, Large=2) or tree-based models that handle integer encoding correctly. Never use for nominal categories with linear models — the model will incorrectly assume Blue (1) is between Red (0) and Green (2) implying an order. One-Hot Encoding: creates a binary column for each category. No false ordinal relationship. Use for nominal categories with linear models, logistic regression, neural networks. Drawback: high cardinality features (thousands of categories) create thousands of columns — use target encoding or embedding instead.

Q43

What is feature scaling? Which algorithms require it?

Answer:

Feature scaling normalises the range of input features so no single feature dominates due to its scale. StandardScaler: $z = (x - \text{mean}) / \text{std}$. Result: mean=0, std=1. MinMaxScaler: $x_{\text{scaled}} = (x - \text{min}) / (\text{max} - \text{min})$. Result: range [0,1], sensitive to outliers. RobustScaler: uses median and IQR — robust to outliers. REQUIRE scaling: KNN (distance-based), SVM (margin maximisation), Neural Networks (gradient descent converges faster), Logistic Regression, PCA. DO NOT require scaling: Decision Trees, Random Forest, XGBoost, Gradient Boosting (split on thresholds, not distances or magnitudes). Critical rule: ALWAYS fit scaler on training data only, then transform both train and test.

Q44

What are the most effective feature selection techniques?

Answer:

Filter methods (model-independent): Select features based on statistical tests. Correlation with target (Pearson for continuous, Spearman for non-linear). Chi-squared test for categorical features. ANOVA F-test. Fast but ignores feature interactions. Wrapper methods (use a model): Recursive Feature Elimination (RFE) — train model, remove weakest features, repeat. Forward selection — start with no features, add best one at a time. Computationally expensive but considers interactions. Embedded methods (built into model training): L1 regularisation (Lasso) sets irrelevant feature weights to zero. Random Forest/XGBoost `feature_importances_` ranks features by their contribution to splits. Most practical: use tree-based feature importance + correlation analysis together.

Q45

Explain target encoding and when to use it.

Answer:

Target encoding replaces each category value with the mean of the target variable for that category. Example: for predicting house price, City='Mumbai' is replaced by the average house price in Mumbai across the training set. Advantage over one-hot encoding: no dimensionality explosion for high-cardinality features (zip codes, user IDs, product SKUs). Risk: severe data leakage if not done carefully — you are using the target to encode features. Fix: use leave-one-out encoding (exclude the current row when computing the mean) or K-fold target encoding (compute mean from out-of-fold rows). Add smoothing: blend category mean with global mean to handle rare categories with few samples.

Q46

What is a neural network? Explain forward and backward propagation.

Answer:

A neural network is a stack of layers of connected nodes (neurons). Each neuron: $\text{output} = \text{activation}(\text{sum}(\text{weights} * \text{inputs}) + \text{bias})$. Forward propagation: input data flows through each layer sequentially, computing activations, until the output layer produces a prediction. Loss is computed between prediction and true label. Backward propagation: the gradient of the loss with respect to each weight is computed using the chain rule, flowing backwards from output to input. Weights are updated by gradient descent: $w = w - \text{learning_rate} * \text{gradient}$. This is the core training loop: forward pass, compute loss, backward pass, update weights.

Q47

What are common activation functions? When do you use each?

Answer:

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$. Most widely used in hidden layers. Fast, no vanishing gradient for positive inputs, sparse activations. Limitation: dead neurons (neurons that always output 0 if input is always negative). Leaky ReLU: $f(x) = \max(0.01x, x)$. Fixes dead neurons. Sigmoid: $f(x) = 1/(1+e^{-x})$. Output range (0,1). Use for binary output layer — outputs a probability. Suffers from vanishing gradient in deep networks. Tanh: range (-1,1). Zero-centred — better than sigmoid for hidden layers in shallow networks. Still has vanishing gradient. Softmax: normalises outputs to sum to 1. Use for multi-class output layer — each output is a class probability.

Q48

What is the vanishing gradient problem? How is it solved?

Answer:

In deep networks, during backpropagation, gradients are multiplied through each layer via the chain rule. Sigmoid and tanh activation functions saturate — their gradients are very small (near 0) for large input magnitudes. Multiplying many small numbers together causes gradients to shrink exponentially as they flow back through deep networks. Early layers barely learn. Solutions: (1) Use ReLU activations — gradient is 1 for positive inputs, never shrinks. (2) Batch Normalisation — normalises layer inputs, keeping gradients in a healthy range. (3) Residual connections (ResNet) — skip connections add gradients directly, bypassing deep paths. (4) He/Xavier weight initialisation — keeps initial gradient magnitudes appropriate.

Q49

What is dropout? How does it prevent overfitting?

Answer:

During each training step, dropout randomly sets a fraction (rate typically 0.2-0.5) of neuron outputs to zero. Each training step uses a different random subset of neurons. During inference, all neurons are active but their outputs are scaled by $(1 - \text{dropout_rate})$. Why it prevents overfitting: (1) Forces the network to learn redundant representations — no neuron can rely on specific other neurons always being present. (2) Effectively trains an ensemble of 2^n different network architectures simultaneously. (3) Reduces co-adaptation of neurons. Typically applied after dense layers, not convolutional layers. Too high a rate slows training. Too low a rate has no effect.

Q50

What is batch normalisation? Why is it useful?

Answer:

Batch normalisation normalises the inputs to each layer across the mini-batch: subtract batch mean, divide by batch standard deviation, then apply learnable scale (gamma) and shift (beta) parameters. Benefits: (1) Accelerates training — reduces internal covariate shift (distribution of layer inputs changes during training), allowing higher learning rates. (2) Regularises the network — reduces dependence on specific weight values, acts like a mild dropout effect. (3) Reduces sensitivity to weight initialisation. (4) Helps with vanishing/exploding gradients. Applied between a linear layer and its activation function. At inference time, uses running mean and variance computed during training (not batch statistics).

Q51

What is transfer learning? How is fine-tuning different from feature extraction?

Answer:

Transfer learning reuses weights from a model trained on a large dataset (e.g. ImageNet, BERT) for a new related task. Feature extraction: freeze all pre-trained layers, add new trainable layers on top, train only the new layers. Fast, requires very little data. Used when new task is similar to original. Fine-tuning: unfreeze some or all pre-trained layers, train with a very low learning rate to slightly adjust weights for the new task. Requires more data and time but achieves higher accuracy. Typical approach: first do feature extraction (warm up new layers), then unfreeze top layers and fine-tune. Full fine-tuning on small datasets risks catastrophic forgetting.

Q52

Explain CNNs and what makes them suitable for image data.

Answer:

Convolutional Neural Networks use convolutional filters (kernels) that slide across the input, computing dot products to detect local patterns. Key components: Convolutional layer — applies filters to detect features (edges, textures, shapes). Learned filters are spatially shared — the same filter detects a pattern anywhere in the image (translation invariance). Pooling layer (MaxPool/AvgPool) — reduces spatial dimensions, provides some position invariance. Fully connected layer — final classification. Why suited for images: (1) Local connectivity captures spatial structure, (2) Weight sharing dramatically reduces parameters vs fully connected (an image with 224x224x3 pixels has 150k inputs — a fully connected layer to 512 neurons needs 77M parameters), (3) Hierarchical feature learning: edges → textures → parts → objects.

Q53

What is an LSTM? What problem does it solve over a vanilla RNN?

Answer:

RNNs (Recurrent Neural Networks) process sequences by maintaining a hidden state passed from one time step to the next. Problem: vanilla RNNs suffer from vanishing gradients over long sequences — they cannot remember long-range dependencies. LSTM (Long Short-Term Memory) solves this with a cell state (long-term memory) and three gates: Forget gate: decides what to remove from cell state (sigmoid * cell state). Input gate: decides what new information to add to cell state. Output gate: decides what part of cell state to output as hidden state. Gates are learned — the LSTM learns when to remember and when to forget. Modern alternative: Transformer with attention mechanism — better at parallelisation and very long-range dependencies.

9. Python & ML Libraries

Q54

What is the difference between `fit()`, `transform()` and `fit_transform()`?

Answer:

`fit(X)`: learns statistics FROM training data (e.g. `StandardScaler` learns mean and std of each feature). Does not modify data. Returns the fitted object. `transform(X)`: applies the transformation using previously learned statistics. `fit_transform(X)`: combines fit and transform in one step. ONLY use on training data. For test/validation data: ALWAYS use `transform()` only (not `fit_transform`). Using `fit_transform` on test data would cause data leakage — the scaler would learn from test statistics, giving the model information it should not have. In sklearn Pipelines, this is handled automatically — the pipeline calls `fit_transform` on training data and `transform` on test data.

Q55

What is a sklearn Pipeline? Why should you always use one?

Answer:

A Pipeline chains preprocessing steps and a final estimator into a single object. Example: `Pipeline([('scaler', StandardScaler()), ('pca', PCA(n_components=10)), ('model', RandomForestClassifier())])`. Benefits: (1) Prevents data leakage — preprocessing is fitted only on training data in cross-validation. (2) Cleaner code — train and predict in one call. (3) Easy deployment — single object encapsulates entire prediction pipeline. (4) Works with `GridSearchCV` — can tune preprocessing and model hyperparameters together. (5) Reproducible experiments. Without Pipeline, manually reapplying preprocessing for each CV fold is error-prone and easy to leak data.

Q56

How do you use pandas groupby for feature engineering?

Answer:

`groupby` splits data into groups and applies aggregation functions. Key pattern for feature engineering: `df['feature'] = df.groupby('group_col')['target_col'].transform('mean')`. The `transform()` function — unlike `agg()` — returns a Series with the same index as the original DataFrame, enabling direct assignment as a new feature. Useful aggregations: mean (average value per group), std (variability within group), count (group size), min/max, nunique (number of unique values). Example: for an e-commerce dataset, `df['user_avg_spend'] = df.groupby('user_id')['order_value'].transform('mean')` creates a feature capturing each user's typical spending behaviour.

Q57

Explain the difference between `map()`, `apply()` and `applymap()` in pandas.

Answer:

`map(func)`: operates element-wise on a Series only. Used for substituting values from a dictionary or applying a simple function to each element. `df['col'].map({'A': 1, 'B': 2})`. `apply(func)`: operates on a Series (element-wise) or DataFrame (row/column-wise via axis parameter). `df.apply(func, axis=0)` applies to columns. `df.apply(func, axis=1)` applies to rows. More flexible than `map`. `applymap(func)`: operates element-wise on an entire DataFrame. Deprecated in newer pandas — use `df.map()` instead. Performance note: avoid `apply()` with complex functions on large DataFrames — it is slow. Prefer vectorised numpy operations or built-in pandas methods where possible.

Q58

How would you identify and handle outliers in a dataset?

Answer:

Detection methods: (1) Z-score: $|z| > 3$ flags values more than 3 standard deviations from the mean. Works for normally distributed data. (2) IQR method: $Q1 - 1.5 \cdot IQR$ to $Q3 + 1.5 \cdot IQR$ defines the normal range. Robust — works for skewed data. (3) Isolation Forest: anomaly detection algorithm that isolates outliers in fewer splits. (4) Visualisation: box plots, scatter plots, histograms. Handling options: (1) Remove — only if you are confident they are data errors. (2) Cap/Winsorize — replace values beyond 1st/99th percentile with those boundary values. (3) Transform — log transform reduces impact of extreme values. (4) Keep and use robust algorithms — tree-based models are relatively robust to outliers.

10. ML System Design & Production

Q59

How would you design an ML system for a spam email classifier?

Answer:

Problem framing: Binary classification — spam vs ham. Data: collect labelled emails (spam/ham), aim for 100k+ examples. Preprocessing: tokenise text, remove stop words, TF-IDF or word embeddings. Features: word unigrams/bigrams, email metadata (sender domain, time sent, link count). Models: start with Logistic Regression + TF-IDF (strong baseline), then try Naive Bayes (fast, good for text), XGBoost, BERT for highest accuracy. Evaluation: F1 score (recall more important — missing spam is less harmful than legitimate emails in spam). Metric: false positive rate must be very low ($< 0.1\%$). Deployment: serve via REST API. Monitor: spam patterns change — retrain weekly on recent emails.

Q60

What is concept drift? How do you detect and handle it?

Answer:

Concept drift occurs when the statistical relationship between input features and the target variable changes over time in production. Types: Sudden drift (new regulations change fraud patterns), gradual drift (user preferences slowly shift), seasonal drift (demand patterns change with season). Detection: (1) Monitor model performance metrics (accuracy, F1) — if they degrade, drift may have occurred. (2) Monitor input data distribution — KL divergence, Population Stability Index (PSI) between training and current data. (3) Statistical tests: Kolmogorov-Smirnov test comparing distributions. Handling: (1) Scheduled retraining on recent data. (2) Online learning — update model continuously. (3) Data windowing — use only recent N months of data. (4) Ensemble of models trained on different time periods.

Q61

What metrics would you monitor for a deployed ML model?

Answer:

Model performance metrics: prediction accuracy, F1, AUC-ROC on incoming labelled data (requires ground truth with some delay). Data quality metrics: feature distribution shift (compare current vs training distributions using PSI or KL divergence), missing value rates per feature, schema violations (unexpected data types or out-of-range values). Operational metrics: prediction latency (p50, p95, p99), throughput (predictions per second), error rates (prediction failures), infrastructure cost per prediction. Business metrics: downstream KPIs affected by the model (conversion rate, fraud caught, customer churn rate). Set up alerts when any metric exceeds a threshold. Use tools like MLflow, Weights & Biases, Evidently AI.

Q62

How do you scale an ML model to serve millions of predictions per day?

Answer:

Optimisation strategies: (1) Model compression — quantisation (reduce float32 to int8, ~4x speedup), pruning (remove near-zero weights), knowledge distillation (train a smaller student model on predictions of large teacher model). (2) Batching — group multiple requests and run one forward pass. (3) Caching — cache predictions for frequent identical inputs. (4) Asynchronous serving — decouple prediction requests from responses for non-real-time use cases. Infrastructure: containerise model with Docker, deploy on Kubernetes for horizontal scaling, use a load balancer, serve behind an API gateway. For very high QPS: use GPU instances, TensorRT for inference optimisation, ONNX for cross-framework deployment. Monitor latency percentiles, not just averages.

Q63

Explain the ML project lifecycle from problem definition to deployment.

Answer:

Phase 1 — Problem framing: Define business objective → translate to ML task (classification/regression/clustering) → define success metric → identify data sources. Phase 2 — Data: collect, explore (EDA), clean (handle missing, outliers, duplicates), label if needed. Phase 3 — Feature engineering: transform raw data into predictive features, handle categorical variables, scale if needed. Phase 4 — Modelling: establish baseline (simple model), experiment with multiple algorithms, tune hyperparameters via cross-validation. Phase 5 — Evaluation: evaluate on held-out test set ONCE, analyse error cases, fairness audit. Phase 6 — Deployment: containerise, build API endpoint, A/B test against current system. Phase 7 — Monitoring: track drift, performance degradation, trigger retraining. This is iterative — insights from later phases feed back to earlier ones.

Q64

What is A/B testing in the context of ML models?

Answer:

A/B testing compares a new model (treatment B) against the current model (control A) by routing live traffic to both simultaneously. Process: (1) Define success metric (conversion rate, revenue per user, CTR). (2) Calculate required sample size for statistical significance. (3) Randomly assign users to A or B — ensure randomisation is user-level (same user always sees same version). (4) Run for sufficient time (at least 1-2 weeks to account for day-of-week effects and achieve target sample size). (5) Analyse results using a statistical test (t-test for continuous metrics, chi-squared for proportions). (6) Check p-value < 0.05 and practical significance (effect size matters, not just statistical significance). Pitfalls: peeking at results early inflates false positive rate, novelty effects, Simpson's paradox.

Q65

What are the ethical considerations in deploying ML systems?

Answer:

Fairness: check if the model performs differently across demographic groups (age, gender, race). Measure equalised odds, demographic parity. Use tools like Fairlearn, AIF360. Transparency: can the model's predictions be explained? Use SHAP values, LIME for local explanations. Regulators and users increasingly demand explainability (GDPR right to explanation). Privacy: does training data contain personal information? Apply differential privacy, data anonymisation, federated learning. Accountability: who is responsible when the model makes a harmful decision? Document model cards. Bias: training data reflects historical biases — a hiring model trained on past hires may perpetuate gender bias. Audit training data for representation. Security: adversarial attacks can fool models with imperceptible input perturbations. Test model robustness.

11. Statistics & Probability for ML

Q66

What is the difference between probability and likelihood?

Answer:

Probability: given fixed parameters, what is the chance of observing this data? $P(\text{data} \mid \text{parameters})$. Example: if a coin is fair ($p=0.5$), what is the probability of getting 7 heads in 10 flips? Likelihood: given fixed observed data, how plausible are these parameter values? $L(\text{parameters} \mid \text{data})$. Example: given we observed 7 heads in 10 flips, how likely is $p=0.5$ vs $p=0.7$? Maximum Likelihood Estimation (MLE) finds the parameters that maximise the likelihood of the observed data. This is exactly what logistic regression and neural networks do when minimising cross-entropy loss.

Q67

What is Bayes theorem? How is it used in ML?

Answer:

Bayes theorem: $P(A|B) = P(B|A) * P(A) / P(B)$. In ML context: $P(\text{class}|\text{features}) = P(\text{features}|\text{class}) * P(\text{class}) / P(\text{features})$. $P(\text{class})$ is the prior — our belief before seeing data. $P(\text{features}|\text{class})$ is the likelihood. $P(\text{class}|\text{features})$ is the posterior — our updated belief after seeing data. Direct use in Naive Bayes classifier. Broader use: Bayesian optimisation for hyperparameter tuning, Bayesian neural networks for uncertainty quantification, probabilistic programming. Bayesian thinking encourages updating beliefs with evidence rather than accepting or rejecting hypotheses.

Q68

Explain the Central Limit Theorem and its importance in ML.

Answer:

The Central Limit Theorem states that the distribution of sample means approaches a normal distribution as sample size grows, regardless of the underlying population distribution. Why it matters for ML: (1) Justifies many statistical tests used in model evaluation (t-tests for comparing models). (2) Underpins confidence intervals on performance metrics. (3) Explains why averaging predictions (ensemble methods) improves accuracy — the average of many estimates is normally distributed around the true value. (4) Justifies assuming normally distributed errors in linear regression. Practical rule: sample means are approximately normal for $n > 30$.

Q69

What is the difference between correlation and causation? Why does it matter in ML?

Answer:

Correlation: two variables move together statistically — knowing one tells you something about the other. Causation: changing one variable CAUSES a change in the other. Classic example: ice cream sales correlate with drowning deaths (both increase in summer) — but ice cream does not cause drowning. Both are caused by a confounding variable (hot weather). In ML: models learn correlations, not causal relationships. A model trained to predict salary might learn that having a female name correlates with lower salary — reflecting historical bias, not a causal relationship. Relying on spurious correlations makes models brittle: they fail when the correlation breaks. Causal ML (do-calculus, instrumental variables) attempts to learn true causal structure.

Q70

What is hypothesis testing? What are Type I and Type II errors?

Answer:

Hypothesis testing evaluates whether observed data provides sufficient evidence to reject a null hypothesis (H_0). Process: define H_0 (no effect) and H_1 (alternative), compute test statistic, compare p-value to significance level α (typically 0.05). If $p < \alpha$: reject H_0 . Type I error (False Positive, α): rejecting H_0 when it is actually true. Concluding a model improvement is real when it is due to chance. Type II error (False Negative, β): failing to reject H_0 when it is false. Missing a real model improvement. Power = $1 - \beta$ = probability of detecting a real effect. In ML: used in A/B testing to determine if model B is significantly better than model A.

12. Advanced ML Topics

Q71

What is semi-supervised learning? Give a real-world example.

Answer:

Semi-supervised learning uses a small amount of labelled data combined with a large amount of unlabelled data for training. The intuition: unlabelled data reveals the structure of the input space, helping the model generalise better. Techniques: self-training (train on labelled data, pseudo-label confident predictions on unlabelled data, retrain), graph-based methods (propagate labels through a similarity graph), consistency regularisation (predictions should be consistent under small perturbations of input). Real-world example: medical image classification where labelling requires expert radiologists (expensive) but unlabelled scans are plentiful. Another example: web content classification where only a fraction of pages are manually labelled.

Q72

What is reinforcement learning? Define agent, environment, state, action and reward.

Answer:

Reinforcement Learning trains an agent to make sequential decisions by maximising cumulative reward. Key components: Agent — the learner/decision-maker (e.g. the RL algorithm). Environment — the world the agent interacts with (e.g. a chess game, a robot's physical world). State (S) — the current situation observed by the agent. Action (A) — what the agent can do in a given state. Reward (R) — feedback signal after each action (positive for good actions, negative for bad). Policy (π) — the agent's strategy: given state S, which action A to take. Value function — expected cumulative future reward from a state. Q-function — expected reward for taking action A in state S. Algorithms: Q-Learning, SARSA, Policy Gradient, PPO, A3C.

Q73

What is the difference between online learning and batch learning?

Answer:

Batch learning: train on the entire dataset at once, produce a final model. To update with new data, retrain from scratch. Suitable when data fits in memory and does not change frequently. Online (incremental) learning: update the model continuously as new data arrives, one sample or mini-batch at a time. Suitable for: streaming data (stock prices, sensor data), data too large to fit in memory, environments where data distribution changes over time (concept drift). sklearn supports online learning via `partial_fit()` method in algorithms like `SGDClassifier`, `MiniBatchKMeans`, `PassiveAggressiveClassifier`. River is a Python library dedicated to online ML.

Q74

What is multi-task learning?

Answer:

Multi-task learning trains a single model on multiple related tasks simultaneously, sharing representations between tasks. The model has shared layers (learning common features) and task-specific output heads. Benefits: (1) Shared representations act as regularisation — learning what is useful for multiple tasks prevents overfitting any single task. (2) Auxiliary tasks can provide useful gradient signal for the main task. (3) More parameter-efficient than training separate models. (4) Tasks can benefit from each other's training data. Examples: NLP model that simultaneously predicts sentiment, entity type, and intent. Self-driving car model that simultaneously detects objects, predicts depth, and segments road. Key consideration: tasks should be related — unrelated tasks can hurt each other (negative transfer).

Q75

What is knowledge distillation?

Answer:

Knowledge distillation trains a smaller student model to mimic the behaviour of a larger teacher model. The student learns from: (1) Hard labels — the actual ground truth. (2) Soft labels — the teacher's output probability distribution (more informative than hard labels because they encode the teacher's uncertainty and inter-class similarities). Temperature scaling softens the probability distribution: $p_i = \exp(z_i/T) / \sum(\exp(z_j/T))$. Higher T produces softer distributions. The combined loss: $L = \alpha * \text{CrossEntropy}(\text{hard_labels}) + (1-\alpha) * \text{KL_divergence}(\text{student_soft}, \text{teacher_soft})$. Benefits: student achieves accuracy close to teacher at a fraction of the size and inference cost. Used to deploy BERT-level NLP models on mobile devices.

Q76

What is federated learning?

Answer:

Federated learning trains ML models across multiple decentralised devices or servers holding local data, without sharing the raw data centrally. Process: (1) A global model is sent to each participating device. (2) Each device trains on its local data and computes model updates (gradients or weight deltas). (3) Only updates — not raw data — are sent to the central server. (4) The server aggregates updates (e.g. FedAvg: weighted average by dataset size). (5) Updated global model is redistributed. Benefits: privacy (raw data never leaves devices), compliance with GDPR, enables learning from sensitive data (medical records, financial data). Challenges: non-IID data across devices, communication cost, adversarial clients sending poisoned updates. Used by Google for Gboard next-word prediction.

Q77

What is SHAP and how does it explain model predictions?

Answer:

SHAP (SHapley Additive exPlanations) assigns each feature a contribution value (Shapley value) for a specific prediction, based on cooperative game theory. The Shapley value of a feature is its average marginal contribution across all possible subsets of features. Interpretation: SHAP value of +5.2 for feature 'income' means that feature pushed the prediction 5.2 units above the baseline. Properties: (1) Local fidelity — correctly explains each individual prediction. (2) Consistency — features that contribute more always get higher SHAP values. (3) Sum of all SHAP values = prediction - baseline. Applications: TreeSHAP for tree models (exact, fast), DeepSHAP for neural networks, KernelSHAP (model-agnostic but slow). Feature importance: $\text{mean}(|\text{SHAP values}|)$ across all predictions gives global importance.

Q78

What is the difference between generative and discriminative models?

Answer:

Discriminative models learn the decision boundary $P(y|x)$ — given features x , what is the probability of class y ? They model the boundary between classes directly. Examples: Logistic Regression, SVM, Neural Networks, Random Forest. Generally better for classification accuracy. Generative models learn the joint distribution $P(x,y) = P(x|y) * P(y)$ — how is the data generated for each class? They can generate new samples and compute $P(x)$ (likelihood of any input). Examples: Naive Bayes, Hidden Markov Models, GANs, VAEs. Can work with missing features. Key insight: discriminative models need only learn the boundary; generative models learn more about the data and therefore need more training data but gain the ability to generate new samples.

Q79

What are GANs? Explain the generator and discriminator.

Answer:

GANs (Generative Adversarial Networks) train two neural networks adversarially: Generator (G): takes random noise z as input, generates fake samples that look like real data. Its goal is to fool the discriminator. Discriminator (D): takes a sample (real or generated), outputs probability that it is real. Its goal is to distinguish real from fake. Training: alternate between training D to correctly classify real vs fake, and training G to maximise D's error on fake samples. Equilibrium: G generates samples indistinguishable from real data; D outputs 0.5 (cannot tell real from fake). Challenges: mode collapse (G generates limited variety), training instability, evaluation difficulty (no clean loss metric). Applications: image synthesis, data augmentation, style transfer, image-to-image translation (pix2pix).

Q80

What is the attention mechanism in transformers?

Answer:

Attention allows a model to focus on relevant parts of the input when producing each output token. Self-attention: each token attends to every other token in the sequence. For each token, compute Query (Q), Key (K), Value (V) matrices by multiplying input embeddings by learned weight matrices. Attention score: $\text{softmax}(Q * K^T / \sqrt{d_k}) * V$. The $\sqrt{d_k}$ scaling prevents vanishing gradients for large dimensions. Multi-head attention: run multiple attention heads in parallel with different learned projections, then concatenate. Each head can attend to different aspects of the input. Why transformers dominate: (1) No sequential processing — parallelisable unlike RNNs, (2) Can model long-range dependencies directly (any token attends to any other), (3) Scales well with more data and larger models.

13. Time Series & Specialised Topics

Q81

What is a time series? What makes it different from regular tabular data?

Answer:

A time series is a sequence of data points indexed in time order. Key differences from tabular data: (1) Temporal dependency — observations are NOT independent; past values influence future values. (2) Cannot shuffle rows — the order matters fundamentally. (3) Train/test split must respect time — test data must come AFTER training data, never random split. (4) Special patterns to model: trend (long-term direction), seasonality (regular periodic patterns), cyclical (irregular multi-year cycles), noise. (5) Stationarity matters — most time series models require the mean and variance to be constant over time. Check with Augmented Dickey-Fuller test; fix non-stationarity by differencing.

Q82

What is ARIMA? Explain each component.

Answer:

ARIMA (AutoRegressive Integrated Moving Average) is a classic statistical time series model. AR (p) — AutoRegressive: current value is a linear combination of the p previous values. Captures momentum: high values tend to follow high values. I (d) — Integrated: number of times the series is differenced to achieve stationarity. Differencing: $y_t' = y_t - y_{t-1}$ removes trend. MA (q) — Moving Average: current value depends on the q previous forecast errors (residuals). Captures shock effects. Seasonal ARIMA (SARIMA) adds seasonal AR, I and MA terms. Choosing p, d, q: use ACF (autocorrelation function) and PACF (partial ACF) plots, or `auto_arima` from `pmdarima` which searches automatically. Evaluation: AIC (lower is better), residual diagnostics.

Q83

What is the difference between MAE, MSE and MAPE for time series evaluation?

Answer:

MAE (Mean Absolute Error): average of $|\text{actual} - \text{predicted}|$. Intuitive, same units as data, robust to outliers. Limitation: does not penalise large errors more than small ones. MSE (Mean Squared Error): average of $(\text{actual} - \text{predicted})^2$. Heavily penalises large errors — useful when big errors are especially costly. RMSE (root MSE) restores original units. MAPE (Mean Absolute Percentage Error): average of $|\text{actual} - \text{predicted}| / |\text{actual}| * 100\%$. Scale-independent — useful for comparing accuracy across different products/series. Limitation: undefined when actual = 0, and biased when actual values are small. SMAPE (Symmetric MAPE) fixes the asymmetry issue. In practice: use RMSE for optimisation, MAPE for business reporting.

Q84

What is exponential smoothing? How does it differ from a simple moving average?

Answer:

Simple Moving Average (SMA): average of the last N observations, weighting all equally. Problem: treats a value from 5 periods ago the same as last period's value. Exponential Smoothing: assigns exponentially decreasing weights to past observations — recent observations get more weight. Formula: $S_t = \alpha * y_t + (1 - \alpha) * S_{t-1}$. Alpha ($0 < \alpha < 1$): smoothing factor. High alpha = more weight on recent data (responsive but noisy). Low alpha = more weight on historical data (smooth but slow to adapt). Holt's method adds a trend component. Holt-Winters adds both trend and seasonality (additive or multiplicative). More flexible and generally more accurate than SMA for forecasting.

Q85

How would you detect anomalies in a time series?

Answer:

Statistical methods: Z-score — flag points more than 3 standard deviations from the rolling mean. IQR method — flag points outside $Q1 - 1.5 \cdot IQR$ to $Q3 + 1.5 \cdot IQR$ on rolling windows. ARIMA residuals — fit ARIMA, flag residuals with high z-scores (model-based anomalies). ML methods: Isolation Forest — isolates anomalies in fewer splits, works well for multivariate series. LSTM Autoencoder — train on normal data, flag high reconstruction errors as anomalies. One-Class SVM — learns a boundary around normal data. Prophet (Facebook) — decomposes series into trend + seasonality + holiday effects, flags points outside confidence intervals. Key insight: define what 'anomaly' means for your use case — point anomaly (single spike), contextual anomaly (unusual given context), collective anomaly (unusual sequence).

14. Interview Strategy & Coding

Q86

How should you structure your answer to a conceptual ML question?

Answer:

Use the IDEA framework: Intuition — start with a one-sentence plain-English explanation of the concept. Definition — give the precise technical definition. Example — provide a concrete, relatable real-world example. Application — mention where it is used in practice and any limitations. Example for 'What is Random Forest?': Intuition: 'It is like asking 500 experts for their opinion and taking a majority vote.' Definition: 'An ensemble of decision trees each trained on a bootstrap sample with random feature subsets.' Example: 'Predicting customer churn — each tree learns from a different sample of customers and votes on whether a customer will churn.' Application: 'Used in fraud detection, medical diagnosis — handles non-linear relationships without feature scaling.'

Q87

Write Python code to implement K-Means clustering from scratch.

Answer:

K-Means steps: (1) Randomly pick K centroids from data points. (2) Assign each point to nearest centroid using Euclidean distance. (3) Recompute centroids as mean of assigned points. (4) Repeat until no reassignments. Key numpy ops: `distances = np.linalg.norm(X[:, np.newaxis] - centroids, axis=2)`; `labels = np.argmin(distances, axis=1)`; `centroids = np.array([X[labels==i].mean(axis=0) for i in range(k)])`. Use `k-means++` init to avoid bad local minima. In sklearn: `KMeans(n_clusters=3, init='k-means++')`.

Q88

Write Python code to calculate precision, recall and F1 from scratch.

Answer:

From confusion matrix values: TP = correctly predicted positives; FP = predicted positive, actually negative; FN = predicted negative, actually positive. Precision = $TP / (TP + FP)$ — of all predicted positives, how many are correct. Recall = $TP / (TP + FN)$ — of all actual positives, how many did we catch. F1 = $2 \cdot P \cdot R / (P + R)$ — harmonic mean balancing both. One-liner check: `tp=sum(1 for a,p in zip(y_true,y_pred) if a==1 and p==1)`; `fp=sum(1 for a,p in zip(y_true,y_pred) if a==0 and p==1)`; `fn=sum(1 for a,p in zip(y_true,y_pred) if a==1 and p==0)`. Verify with `sklearn.metrics.precision_recall_fscore_support(y_true, y_pred)`.

Q89

What questions should you always ask before starting any ML problem?

Answer:

Business context: What is the business objective? How will the model's output be used? What happens if the model is wrong? Data: What data is available? How much? How clean? Is it labelled? How was it collected — any sampling bias? Task definition: Is this classification, regression, clustering or something else? What is the target variable? Evaluation: What metric should optimise for? What is the acceptable level of false positives vs false negatives? What is the current non-ML baseline we need to beat? Constraints: What are the latency requirements? Real-time or batch? What compute is available? Deployment: How will the model be deployed? Web API, mobile, edge device? How will it be monitored and retrained? Asking these questions in an interview shows engineering maturity — interviewers reward candidates who think beyond the algorithm.

Q90

What are common ML interview mistakes and how do you avoid them?

Answer:

Mistake 1: Jumping to a complex model without a baseline. Always establish a simple baseline (majority class, linear model) before proposing deep learning. Mistake 2: Ignoring data issues. In practice, data quality matters more than algorithm choice. Always discuss data cleaning and feature engineering. Mistake 3: Using accuracy on imbalanced data. Always check class distribution first and choose appropriate metrics. Mistake 4: Not mentioning overfitting risks. For every complex model proposed, mention regularisation and validation strategy. Mistake 5: Ignoring interpretability. Many production ML systems require explainable predictions — mention SHAP, feature importance. Mistake 6: Not knowing complexity. Know the training and inference time complexity of the algorithms you propose. Mistake 7: No deployment plan. Always finish with how you would deploy, monitor and retrain the model.

15. Bonus: Real Interview Questions from Top Companies

Q91

Google: How would you build a YouTube recommendation system?

Answer:

Frame as two-stage retrieval + ranking. Stage 1 Candidate Generation: use collaborative filtering or a two-tower neural network to retrieve ~100 candidate videos from millions. User tower encodes watch history, demographics. Video tower encodes content features. Dot product similarity for retrieval. Stage 2 Ranking: train a deep neural network on the 100 candidates with richer features — watch time, click probability, freshness, diversity. Optimise for watch time (regression) or click-through rate (classification). Key challenges: feedback loops (popular videos become more popular), user cold start (new users have no history), content cold start (new videos have no watch data). Evaluation: offline A/B test on watch time per session. Address diversity — avoid recommending only one category.

Q92

Amazon: How would you detect fraudulent transactions in real time?

Answer:

Frame as binary classification with extreme class imbalance (fraud is < 0.1% of transactions). Features: transaction amount (z-score vs user history), merchant category, time of day, location change speed (impossible travel), device fingerprint, velocity features (transactions per hour). Model: XGBoost or LightGBM for initial model, LSTM for sequential transaction patterns. Handle imbalance: `class_weight='balanced'`, SMOTE, or threshold tuning to maximise recall. Real-time requirement: model must respond in < 100ms. Deploy via REST API, precompute user-level features in a feature store (Redis). Evaluation: recall at fixed precision (catch 90% of fraud with < 1% false positive rate). Monitor for concept drift — fraud patterns change constantly.

Q93

Meta: Design a system to detect hate speech in social media posts.

Answer:

Input: text posts (multilingual, noisy, abbreviations, slang). Pipeline: preprocessing (normalise text, handle emojis, slang dictionaries). Model: fine-tuned multilingual BERT (mBERT or XLM-R) for classification. Labels: hate speech, offensive but not hateful, neither. Challenges: context matters (quoting hate speech to condemn it is different from promoting it), sarcasm, evolving slang, false positives impact free speech. Evaluation: human review of borderline cases, precision-recall tradeoff (higher recall = more false positives = over-censorship). At scale: two-stage — fast keyword/regex filter, then ML model on flagged content. Human reviewers for high-stakes decisions. Bias audit: ensure model does not disproportionately flag content from specific demographic groups.

Q94

Microsoft: How would you build an autocomplete system?

Answer:

Approaches by latency and accuracy tradeoff. Simple approach: n-gram language model with frequency counts from query logs. Fast, interpretable, no GPU needed. Better approach: prefix tree (trie) storing popular completions, filtered by user context and personalisation. ML approach: transformer language model (GPT-style) fine-tuned on query logs to predict next tokens. Personalisation: weight suggestions by user's past queries. Ranking: score completions by LM probability, query frequency, recency, personalisation score. Latency requirement: must respond in < 50ms as user types. Solution: pre-compute completions for common prefixes, cache in Redis, use quantised model for rare prefixes. Evaluation: completion acceptance rate, keystrokes saved per query.

Q95

Netflix: How would you measure the success of a new recommendation algorithm?

Answer:

Define north star metric: long-term member retention (subscription renewal) — not just clicks. Short-term proxy metrics: play rate (recommended item actually played), watch completion rate (did they finish?), satisfaction rating. Why not CTR alone: optimising for clicks creates clickbait. A/B test design: randomly assign members to control (old algorithm) or treatment (new algorithm) — ensure randomisation is at member level. Run for minimum 2 weeks (capture weekly usage patterns). Statistical test: t-test on mean watch time per member per day, chi-squared on renewal rate. Guardrail metrics: ensure diversity of content types is not harmed. Long-term holdout: track 6-month retention — a recommendation that maximises short-term engagement may cause burnout. Report both statistical and practical significance.

Q96

Uber: How would you predict estimated time of arrival (ETA)?

Answer:

Features: distance and route (from maps API), time of day, day of week, weather, historical travel times on each road segment, current traffic (real-time from GPS pings), special events, driver behaviour score. Model: gradient boosting (XGBoost/LightGBM) for initial model — handles mixed features well. Deep learning: graph neural network over the road network, or sequence model over waypoints. Training data: completed trips with actual travel times as labels. Evaluation: MAE and MAPE on held-out trips by region and time of day. Cold start: for new routes, rely on map-based estimates. Live retraining: retrain daily on recent trips to capture seasonal patterns. Key insight: overestimating ETA is better than underestimating — passengers prefer pleasant surprises over late arrivals. Bias the model slightly toward higher estimates.

Q97

What is the difference between a discriminative model and a generative model in interview context?

Answer:

Discriminative: learns the decision boundary $P(y|x)$ — directly models which class given input. Examples: Logistic Regression ($P(\text{class}|\text{features})$), SVM (maximises margin), Neural Networks. Better classification accuracy with sufficient data. Generative: learns how data was generated $P(x,y) = P(x|y)P(y)$. Can generate new samples, works with missing features, better with small data. Examples: Naive Bayes (models $P(\text{features}|\text{class})$), VAEs, GANs. Interview tip: when asked 'how would you approach this problem', always ask whether you need to classify only (discriminative) or also understand data structure / generate samples (generative). Most production classification tasks: discriminative. Data augmentation, anomaly detection, density estimation: generative.

Q98

How do you ensure reproducibility of ML experiments?

Answer:

Code: version control with Git, commit hash in experiment logs. Dependencies: pin exact package versions in requirements.txt or conda environment.yml, use Docker containers for full environment reproducibility. Data: version your datasets (DVC — Data Version Control), log dataset hash/checksum with each experiment. Randomness: set all random seeds (numpy, Python random, torch, sklearn) — `np.random.seed(42)`, `torch.manual_seed(42)`. Log every hyperparameter and configuration, not just the ones you changed. Experiment tracking: use MLflow, Weights & Biases or DVC to log: hyperparameters, metrics, artifacts, code version, data version. Hardware: note GPU type — floating point operations can differ slightly between GPU models. Document environment: Python version, CUDA version, key library versions.

Q99

What is the Transformer architecture? Why did it replace RNNs for NLP?

Answer:

Transformer (Vaswani et al. 2017): encoder-decoder architecture using only attention mechanisms, no recurrence. Key components: Multi-head self-attention (each token attends to all others), positional encoding (injects sequence position since attention is position-invariant), feed-forward layers, layer normalisation, residual connections. Why it replaced RNNs: (1) Parallelisation — RNNs process tokens sequentially (cannot parallelise training); Transformers process all tokens simultaneously — massive training speedup on GPUs. (2) Long-range dependencies — attention directly connects any two positions in $O(1)$ operations; RNNs need $O(n)$ steps to propagate information across n positions. (3) Scalability — performance keeps improving with more data and parameters (scaling laws). Foundation for BERT (encoder), GPT (decoder), T5 (encoder-decoder).

Q100

What would you do if your model performs well in development but poorly in production?

Answer:

This is a classic train-serve skew problem. Investigation steps: (1) Check data distribution — compare feature distributions between training data and production data using PSI or KL divergence. If they differ, model trained on stale or unrepresentative data. (2) Check feature pipeline — are features computed identically during training and serving? A common bug: computing age as `(current_date - birth_date)` where `current_date` is hardcoded during training but dynamic in production. (3) Check for label leakage — was a target-correlated feature accidentally included in training? (4) Check for concept drift — has the real-world relationship between features and target changed? (5) Check logging — are production predictions being logged correctly for analysis? Fixes: retrain on recent data, fix feature pipeline bugs, add more monitoring, implement drift detection alerts.

Keep learning at learnmlacademy.com

50+ in-depth tutorials • Python code examples with output • Visual diagrams •
Quiz sections • Deep Learning • Interview prep • Always free